$ cat portfolio.md

Jihwan Choi

Software Engineer / MLOps Engineer

jhwan333@gmail.com · linkedin.com/in/jihwan-choi · github.com/jhwanchoi · jhwanchoi.github.io

₩448M	6mo→1.5mo	99.97%	80M/day
SAVED VS AWS	VALIDATION TIME	SUCCESS RATE	LOG EVENTS

01 Introduction

A software engineer who designs and operates an MLOps inference & evaluation platform for autonomous-driving AI development. I co-designed a shared GPU cluster (RKE2 Kubernetes, 60+ heterogeneous GPUs) with another engineer and led the scheduler & queue policy design, and I hold sole ownership of the inference/evaluation workflow (SVEvalFlow: PRISM, Inferit, MTBF Validation).

I cut a 20,000-hour-scale validation that previously took 6+ months on a single machine down to 1.5 months via distributed/parallel processing (173,101 clips, 99.97% success rate), and led a cost-driven On-Premise transition that saved ₩448M (55%) vs AWS. By analyzing abnormal workloads I prevented ~₩100M/month in overbilling, and I operate a log-analysis system processing 80M daily events — consistently turning technical solutions into measurable business value.

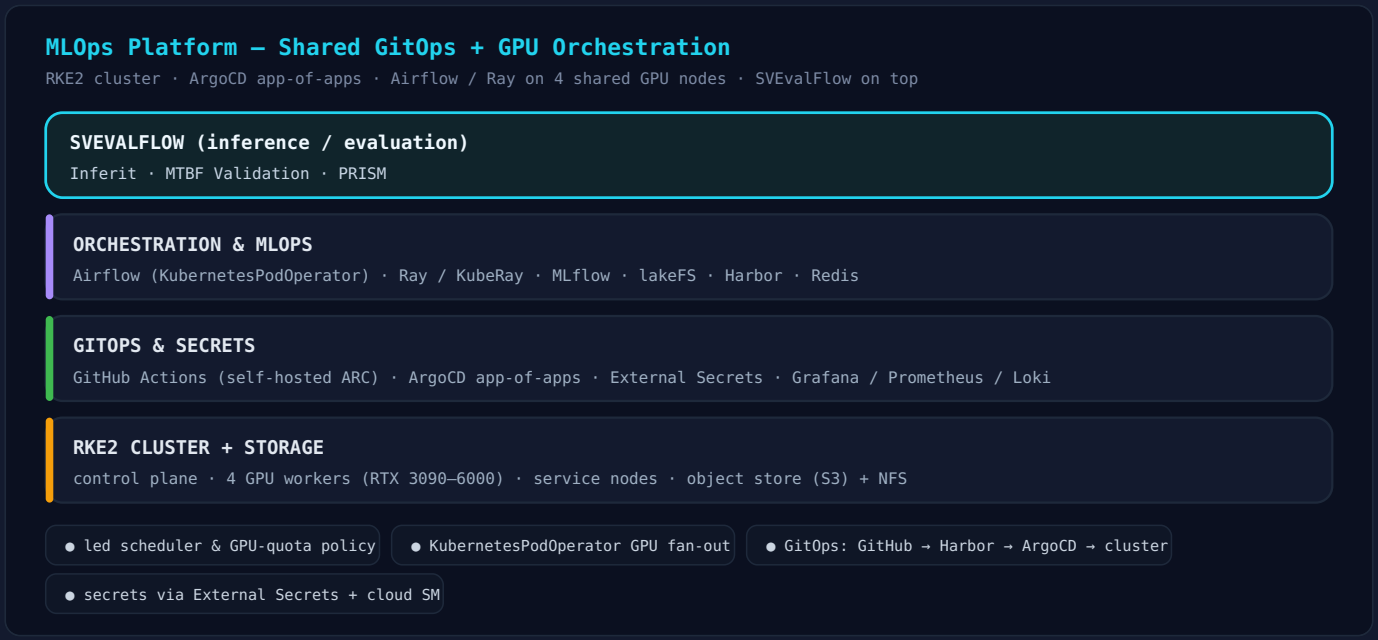
Working with teams such as the Validation Team and Data Utilization Team, I analyze requirements and own new services from planning through architecture, development, deployment, and operation (design-only or end-to-end, depending on the project). I have hybrid-infrastructure experience spanning On-Premise and AWS, and I value documenting complex technical problems and sharing them with the team. Most recently I build AI-native developer tools (LLM Wiki, Multi-Model Debate, Bedrock cost observability) and evangelize them internally.

02 Projects

MLOps Platform — Shared GPU Cluster Co-Design & SVEvalFlow Owner

Oct 2025 – Present

Co-design/build of the in-house MLOps cluster and ownership of the inference/evaluation workflow



▷ **Background & Problem:**

- Inefficiency from manual management across the full model training–inference–evaluation cycle

- Lack of experiment tracking and reproducibility; insufficient training/evaluation data versioning
- Need for unified management of a heterogeneous GPU cluster for distributed training/inference

▷ **Shared cluster architecture (co-designed with another engineer):**

- Co-designed and built a hybrid (GPU/CPU) cluster on RKE2 Kubernetes; 60+ heterogeneous GPUs (RTX 3090/4080/4090/5090, RTX 6000)
- Service routing via NGINX Gateway Fabric, TLS via Cert-Manager, shared storage via NFS CSI, bare-metal load balancing via MetalLB
- Led scheduler & queue policy design: nodeSelector + per-GPU-type tolerations for heterogeneous workload distribution, Resource Quota & Priority Class to guarantee critical-job priority
- Integrated 15+ core components: Ray/KubeRay, MLflow, Airflow, ArgoCD, KServe, Harbor, Nexus, etc.
- Collaborate on the training workflow (SVDeepFlow, scheduler/queue policy only); sole ownership of the inference/evaluation workflow (SVEvalFlow)

▷ **Technology selection rationale:**

- RKE2: better suited than EKS for directly managing On-Premise GPU servers, with strong security defaults
- Ray/KubeRay: Python-native API improves ML-engineer accessibility; RayJob CRD enables Kubernetes-native batch training/inference
- KServe: Knative-based Scale-to-Zero cuts idle GPU cost vs running TorchServe/Triton directly
- ArgoCD: intuitive UI and multi-cluster support vs Flux CD, with native Helm/Kustomize support

▷ **Challenges & resolutions:**

- Frequent abnormal pip install timeouts on a CPU node pool → stood up a Nexus proxy registry for dependency caching
- protobuf version conflicts between Ray worker pods causing RayJob failures → pinned versions and isolated dependencies with a Docker multi-stage build
- Token-prompt issues on SSH clone of in-house private repos → switched to HTTP clone + env-variable auth

▷ **Three CI/CD pipelines:**

- Pipeline 1 (via Airflow): GitHub → Image Build → Airflow DAG → ArgoCD → RayJob distributed training
- Pipeline 2 (ArgoCD direct): GitHub → Image Build → ArgoCD → RayJob immediate execution
- Pipeline 3 (inference deploy): GitHub → ArgoCD → KServe/Knative serverless inference (Scale-to-Zero)

▷ **Monitoring & artifact management:**

- Unified Grafana + Prometheus + Loki monitoring dashboard
- Harbor private container/Helm-chart registry, Nexus artifact store (dependency caching)
- MLflow experiment tracking and model artifact management

```
Architecture: GitHub → GitHub Actions → Harbor Registry → (Airflow DAG / ArgoCD GitOps / KServe) → Ray / KubeRay (distributed GPU) → MLflow, NFS, Grafana
```

▷ **Results:**

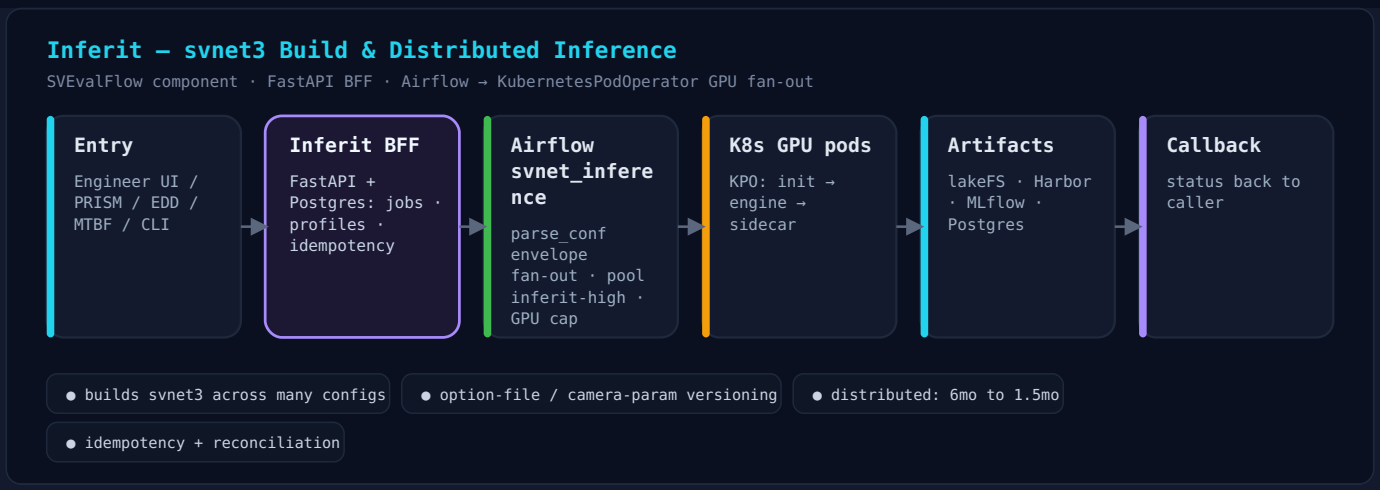
- Cluster: co-designed a 60+ heterogeneous-GPU shared cluster; led scheduler & queue policy
- Pipelines: built 3 automated training/inference/deploy CI/CD pipelines
- Components: integrated 15+ open-source MLOps components on Kubernetes

Tech: Python, Kubernetes (RKE2), Ray, KubeRay, KServe, Knative, MLflow, Airflow, ArgoCD, Harbor, Nexus, GitHub Actions, Helm, Grafana, Prometheus, Loki, NFS CSI, MetalLB

Inferit — svnet3 Build/Inference Acceleration & Artifact Management

2025 — Present

svnet3 build artifact management, distributed/parallel inference, and evaluation automation (core of SVEvalFlow)



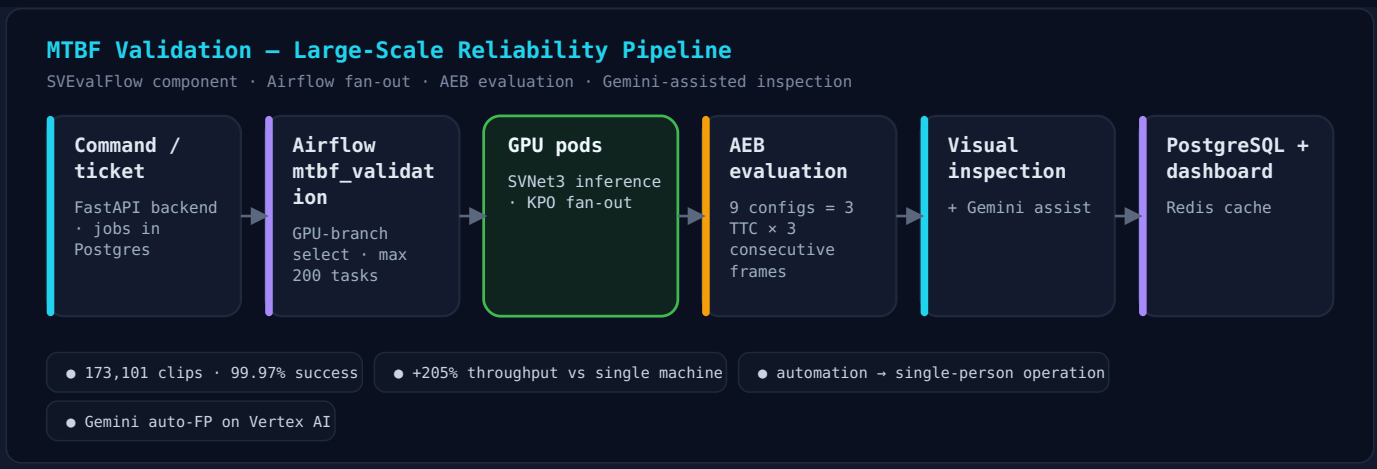
- **Background & Problem:**
  - Fragmented build arguments across svnet3 (in-house AD vision model) SW versions made automation hard
  - Customer option files (e.g., camera parameters) attached to tickets were not managed systematically
  - Single-machine inference took excessive time for large-scale validation/re-inference
  - Manual copy & paste of evaluation outputs; no version tracking
- **Build artifact/config management:**
  - Built version/config/artifact management for building svnet3 across many configs
  - Dual Docker image design (internal-automation vs customer) → internal image includes a unified entrypoint and ini templates, resolving per-version argument fragmentation
- **Customer option-file versioning:**
  - Versioned customer option files (e.g., camera parameters), eliminating the 1–2 days of engineer↔PM search/ communication load
- **Distributed/parallel inference acceleration:**
  - Parallelized single-machine inference across cluster GPUs → acceleration proportional to GPUs per node
  - Extreme case: a 20,000-hour-data MTBF validation went from an estimated 6+ months → 1.5 months
- **Evaluation automation (EDD integrated):**
  - Manual copy & paste of evaluation outputs (xlsx) → automatic metric logging to MLflow for per-version comparison
  - Versioning of evaluation data (256GB+) → Git-like commit/branch snapshots via lakeFS
- **Technology selection rationale:**
  - lakeFS: Git-like branching and an S3/HCP-compatible interface vs DVC; compatibility with existing HCP storage was the key criterion
  - MLflow: open-source and deployable in-house vs Weights & Biases; runs integrated with the MLOps cluster
- **Results:**
  - Inference acceleration: 20,000h validation est. 6+ months → 1.5 months (vs single machine)
  - Search load: customer option-file versioning eliminated 1–2 days of communication
  - Version compatibility: 5+ major release versions supported; automated inference per issue ticket

Tech: Python, Ray, MLflow, lakeFS, Docker, Kubernetes

MTBF Validation — Large-Scale svnet3 Reliability Validation

2022 – 2025

Automated large-scale reliability validation for the svnet3 autonomous-driving model (SVEvalFlow)



- **Background & Problem:**
  - Need for MTBF (Mean Time Between Failures) validation over 20,000 hours of driving-video data (PB-scale)
  - Mixed resolutions (2MP, 3MP, 8MP) and clip lengths (30s–5min) made accurate resource estimation hard
  - Limited compute and budget with a short completion deadline
  - Previously only small-scale validation was feasible; large-scale validation could no longer be deferred
- **Distributed processing architecture:**
  - Accelerated validation using Inferit's distributed/parallel inference
  - Automated the Build → Inference → KPI pipeline via Airflow DAGs
  - 7-type data validation pipeline (FormatStatus, FrameDrop, ValueRange, etc.)
  - Real-time monitoring dashboard + integrated Evaluation & Visual Inspection UI (video/statistics around flagged frames)
  - Introduced Gemini-assisted review to reduce repetitive verification load — humans make the final call
- **Technology selection rationale:**
  - Airflow: DAG-based visual pipeline management vs Celery/Luigi; web UI lets non-developers (Validation Team) monitor progress
  - Kubernetes distributed processing: better dynamic scheduling and resource isolation (namespace/RBAC) than Slurm, plus existing in-house K8s infra
- **Challenges & resolutions:**
  - Job-scheduling imbalance on a heterogeneous-GPU cluster → optimized distribution via nodeSelector + per-GPU-type tolerations
  - Transfer bottleneck between On-Premise NFS ↔ S3 for large data → established a hybrid storage strategy via cost analysis
  - GPU pod resource contention during Airflow DAG runs → guaranteed validation-job priority via Resource Quota + Priority Class
- **Infrastructure transition (Cloud → On-Premise):**
  - Proposed and executed an On-Premise transition based on cost analysis (AWS ~~₩820M~~ vs On-Premise ~~₩370M~~, 55% saved)
  - Ran benchmarks and spec proposals for GPU hardware (RTX 3090/4080/4090, etc.) to support purchasing decisions
  - Leveraged HCP Object Storage (one-time cost vs S3's recurring charges)
- **GPU cluster operation:**
  - Managed 60+ GPUs on a Rancher-based K8s cluster
  - CUDA setup, GPU job testing, fault response and troubleshooting

Architecture: `Git Repo (svnet3) → Airflow Scheduler → K8s Pods (GPU ×60+) → (Build / Inference / KPI) → PostgreSQL / HCP (results)`

- **Results:**
  - Throughput delivered: 173,101 clips processed, 99.97% success rate
  - Throughput gain: 205% more clips vs a single machine
  - Processing time: 67.3% lower vs single machine (est. 6+ months → 1.5 months)

- Cost savings: AWS ₩820M → On-Premise ₩370M (55% saved)
- Operational efficiency: previously feasible only at small scale; now operated by 1 person via automation

Tech: Python, FastAPI, Airflow, Kubernetes, PostgreSQL, HCP Object Storage, Rancher

PRISM — Customer Jira-Integrated ISP Conversion & Re-simulation Pipeline

Nov 2025 –

Jira-based ISP conversion and Re-simulation automation pipeline (SVEvalFlow)

- **Background & Problem:**
  - Manual handling of customer Jira-ticket-based ISP conversion requests
  - Difficulty tracking conversion status; no linkage with Re-simulation work
- **Event-driven architecture:**
  - Receive ticket events via Jira Webhook; asynchronous processing via AWS SQS + Lambda
  - Slack notifications on status change; processing-history dashboard
- **Infrastructure as Code & integrations:**
  - Provision VPC, EC2, SQS, Lambda via Terraform; dev/prod environment separation
  - In-house ISP conversion tool integration with callback-based status updates
  - Inference requests to Inferit and result-history management
- **Re-simulation pipeline (Phase 2, in dev):**
  - Did the initial development, now technical lead tracking/managing another org's developer
  - After ISP conversion, auto-trigger svnet3 Re-simulation via Airflow DAG
  - Dual-version inference (issue + latest) via SWIT Docker Images
  - Processing scopes: CONVERSION_ONLY / FULL (Download→ISP→Upload→Resim) / RESIM_ONLY
  - Manual trigger from PRISM Dashboard; Resim results auto-posted as Jira comments
- **Technology selection rationale:**
  - Event-driven (Jira Webhook → SQS → Lambda): ISP requests are asynchronous and irregular, so serverless is cost-efficient — Lambda gives zero idle cost vs always-on ECS
  - Terraform: multi-cloud support and HCL readability vs CloudFormation; workspaces for dev/prod separation
- **Challenges & resolutions:**
  - Duplicate Jira webhook events → idempotency via SQS FIFO queue deduplication IDs
  - Auto-feedback to Jira tickets → callback pattern: the in-house ISP tool posts status on completion, retries on failure, and Slack-alerts the owner after 3 failures

Architecture: Jira Ticket → Webhook → AWS SQS → Lambda → PRISM BE (FastAPI) → (PostgreSQL / in-house ISP tool / Slack)

- **Results:**
  - Automated processing: ~150 ISP conversion requests/month automated (deployed/operating)

- Processing time: 80% faster than manual
- Infrastructure: automated environment provisioning via Terraform IaC

Tech: Python, FastAPI, PostgreSQL, AWS (SQS, Lambda), Terraform, Jira Webhook, Airflow

WLS — Workload Logging Service

2023 – 2025

Labeling-work log analysis and outsourcing-settlement automation system

▷ **Background & Problem:**
- No system for analyzing labeler workloads; errors from manual verification during outsourcing settlement; excess cost from abnormal workloads

▷ **Large-scale log processing & analysis:**
- Log collection via AWS SQS; storage/analysis of 80M daily events in MongoDB; MongoDB Atlas migration for stability
- State-machine-based workload analysis: 30+ state definitions to analyze labeling patterns and auto-detect abnormal workloads
- Settlement automation via aggregation pipeline; support for new features (OD, SOD, TSTLD, RMD)
- Used WLS log analysis to pinpoint workflow bottlenecks → fed UI/UX improvements and the ALAS model, improving labeling efficiency by 20%

▷ **Technology selection rationale:**
- MongoDB: schemaless structure fits 30+ state unstructured logs vs PostgreSQL; document-level locking favorable for the 80M/day write-heavy workload
- AWS SQS: minimal management overhead vs Kafka, fitting the log-collection profile

▷ **Challenges & resolutions:**
- Write degradation processing 80M/day on a single MongoDB instance → Atlas migration with sharding/replica sets, backup-cycle optimization (12 → 20 weeks)
- Detecting abnormal patterns → classified via a State Machine over 30+ states (Idle, Active, Paused, Reviewing, Submitted, AFK, etc.)
- Complex per-vendor, per-feature cost calc → automated aggregation via $group/$unwind/$lookup chaining

Architecture: Labeling Tool → AWS SQS → WLS BE (Django) → MongoDB (80M events/day) → (State Machine / Aggregation / Billing Report)

▷ **Results:**
- Overbilling prevented: ~₩100M/month (settlement-audit confirmed; ~₩600M cumulative in H1 2024)
- Daily throughput: 80M events; 30+ State Machine; labeling efficiency +20% (UI/UX improvements linked to ALAS)

Tech: Python, Django, MongoDB, AWS SQS, EC2, MongoDB Atlas

ALAS — Auto Labeling Assistant Service

2024

Auto object-generation model operation service

- **Problem & solution:**
  - Excessive labeler workload (bottlenecks identified via WLS log analysis); need to support multiple models
  - FastAPI-based microservice, AWS SNS/SQS messaging; AWS ECS → EKS migration design, CDK-based IaC
  - Model support: Ground Fitting OD/FSD, Occlusion Score
- **Technology selection & challenges:**
  - FastAPI: native async/await optimizes I/O wait on inference requests, with Pydantic-based auto API docs
  - SNS/SQS fan-out: a single labeling request needs concurrent inference by several models (SNS → multiple SQS)
  - Response-time variance across models → async fan-out then aggregation, timeout set to the slowest model
  - Converted the CDK-defined ECS stack to a Helm chart for EKS, keeping CDK only for the infra layer
- **Results:**
  - ~35% reduction in manual annotation workload; multi-model operation; operational efficiency via ECS → EKS migration

Tech: Python, FastAPI, AWS ECS/EKS, SNS/SQS, CDK

Other Projects

2022 — 2025

- **IGS (Ingestion Service, 2024):** Planning and architecture design for an external-customer data collection/conversion service; REST API/DB schema design, Airflow-based conversion-pipeline architecture (design-only).
- **SURF Compass Service (2025):** Planning and design for a data-collection-device monitoring service (design-only); FastAPI-based REST API/DB design.
- **RNS (Release Note Service, 2023):** Automated Jira-ticket-based release-note generation via NestJS + Atlassian API; cut authoring time 50% (4h → 2h).
- **SLT (Smart Labeling Tool, 2022):** 2D LD labeling tool in C++/MFC, improved productivity by 30%.

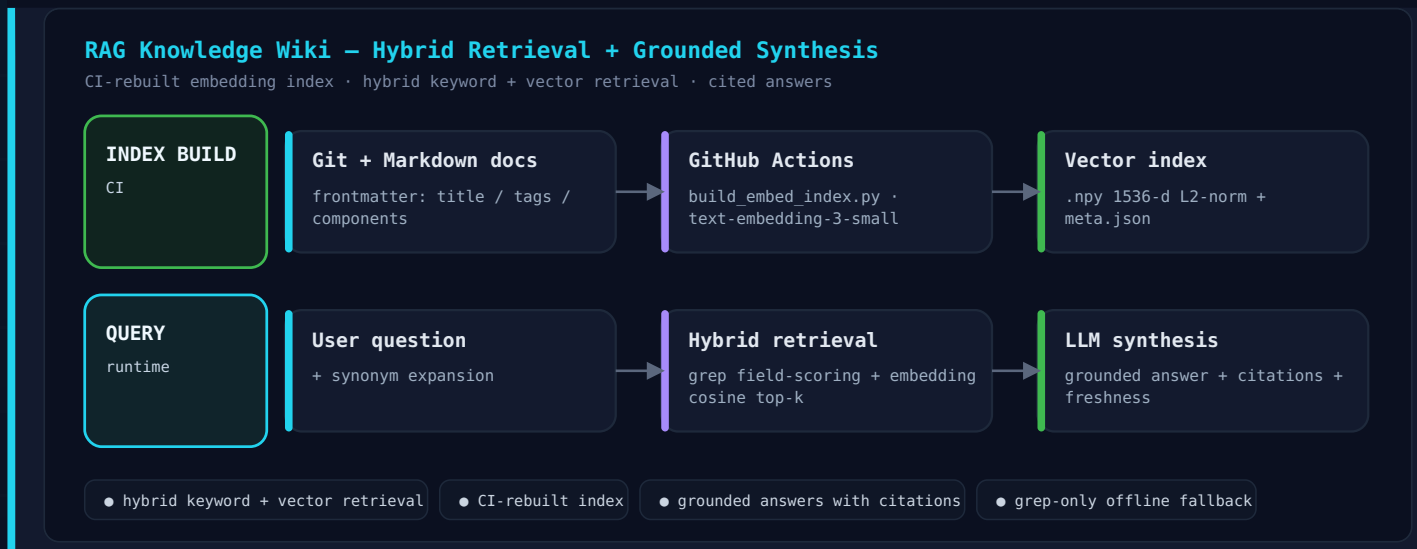
03 Tech Stack Summary

Backend	Python · Django · FastAPI · NestJS
Database	PostgreSQL · MongoDB · Redis
Cloud	AWS (EC2, EKS, ECS, SQS, SNS, Lambda, S3, Bedrock)
Infrastructure	Kubernetes (RKE2) · Docker · Terraform · CDK · Rancher
Workflow	Airflow · Celery
Monitoring	Grafana · Prometheus · Loki
ML / MLOps	Ray · KubeRay · MLflow · KServe · Knative · lakeFS
CI / CD	ArgoCD · GitHub Actions · Harbor · Helm
AI / LLM	Claude Code · Gemini CLI · Codex · MCP · AWS Bedrock · n8n

04 AI-Native Development & Internal Evangelism

LLM Wiki — MLOps Ops-Knowledge Automation

2025 — Present



- Removed Confluence/external dependencies and redesigned around a Git + Markdown LLM Wiki.
- Key features: natural-language authoring (frontmatter, commit/push automation), ops-doc search, document-grounded Q&A, quality lint (stale/schema/orphan/broken-link checks), manual sync.
- Data-freshness strategy: local grep search by default + fresh-sync prompt after a time threshold + pull→commit→push on write.
- **Purpose:** turn scattered individual MLOps ops knowledge into a team asset and remove usage friction.

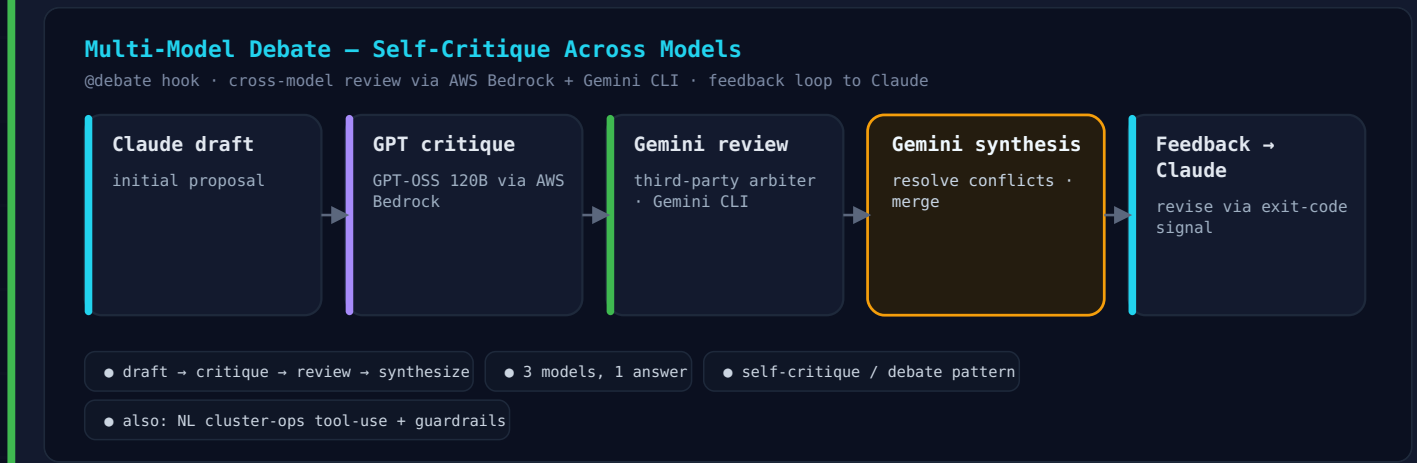
Cluster-ops Claude Plugin

2025

- Exposed MLOps cluster operations utilities (diagnostics, storage, etc.) as slash commands.
- Separated documentation features from ops utilities by role for maintainability.

Multi-Model Debate Hooks

2025



- Designed a cross-review workflow where multiple LLMs check each other to reduce hallucination/bias.
- @debate: Claude draft → GPT critical analysis → Gemini third-party review → Claude synthesis.
- Applied to quality-critical work like design reviews and important document reviews (latency/cost increase acknowledged as a trade-off).
- Public: github.com/jhwanchoi/dotfiles, LinkedIn post.

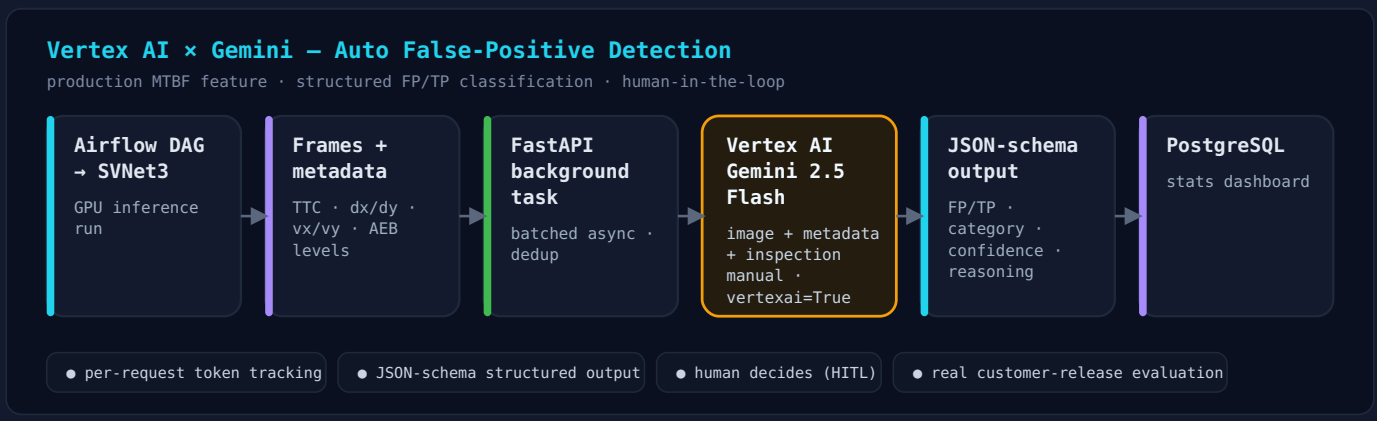
AWS Bedrock Usage/Cost Observability

2025

- ▷ Built monitoring infra for cost visibility as in-house LLM usage grew.
- ▷ Token usage and cost estimation; operated via CloudWatch dashboard and Slack alerts.
- ▷ Manages AI adoption with observable cost/usage operations rather than stopping at adoption.

MTBF Visual Inspection AI Assist

2025



- ▷ Eased the burden of humans reviewing every result in large-scale inference validation using Gemini.
- ▷ Gemini is not the final judge — used to pre-select cases to check and assist review (humans decide).

Workflow Automation Prototypes (n8n)

2025

- ▷ Prototyped AI work-automation connecting Slack, GitHub, monitoring systems, and external APIs.
- ▷ PR summary/first-pass review assist, tech newsbot, first-response cluster monitoring assist, recruiting assist (résumé summary, interview-question drafts — for human review).

Internal Evangelism

2025 — Present

- ▷ Hosted internal AI-driven development meetings; shared trends in modern AI dev tools (Claude Code, Gemini CLI, Codex; Plugin/MCP/Skill).
- ▷ Contributed to spreading an AI-native development culture.

Contact

Email: jhwan333@gmail.com **GitHub:** github.com/jhwanchoi **LinkedIn:** linkedin.com/in/jihwan-choi-717554233/